

MODULE 8

MongoDB Best Practices, Security, and Troubleshooting

Model, tune, secure, recover and diagnose training_store — and decide whether it is ready for production.

Data modeling

Performance

Security

Backup and
recovery

Troubleshooting

LEARNING OBJECTIVES

What you will be able to do by the end of this module.



BY THE END OF THIS MODULE, YOU CAN

1. Apply modeling, validation and evolution rules
2. Tune queries, pipelines and indexes with explain
3. Secure access with auth, roles, TLS, network
4. Plan backups and prove restores (RPO, RTO)
5. Monitor health and troubleshoot by layer
6. Judge training_store's production readiness



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

IS

the model safe to grow?

HOW

do we prove it is fast?

WHO

may do what — and from where?

CAN

we recover from a mistake?

WHAT

is wrong right now?

ARE

we ready to go live?

KEY TAKEAWAY Production-ready is a claim you prove with evidence — not a feeling you have.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 Schema validation

\$jsonSchema rules the server checks on every insert and update — required fields, types, enums.

2 Query shape

A query's filter, sort and projection. Indexes are designed around shapes, not single queries.

3 ESR rule

Order compound-index keys Equality first, then Sort, then Range.

4 Least privilege

Give each identity only the actions and collections its job needs — nothing more.

5 TLS

Transport Layer Security: encrypts traffic between clients, servers and cluster members.

6 RPO and RTO

How much data you may lose, and how long recovery may take — both are time targets.

7 Restore test

Proving a backup by restoring it to an isolated target and checking counts, samples and indexes.

8 Runbook

Written steps for one alert: checks, fixes, escalation, rollback and verification.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

BSON

Binary JSON

COLLSCAN

Collection Scan

CPU

Central Processing Unit

DBA

Database Administrator

ESR

Equality, Sort, Range

ID

Identifier

IP

Internet Protocol

IXSCAN

Index Scan

RBAC

Role-Based Access Control

RPO

Recovery Point Objective

RTO

Recovery Time Objective

TLS

Transport Layer Security

URI

Uniform Resource Identifier

UTC

Coordinated Universal Time

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 7 TO MODULE 8

The cluster stays up and scales — now make it safe to run for real.

MODULE 7 GAVE US

- ✓ Replica sets: primary and secondaries
- ✓ Elections and automatic failover
- ✓ Read and write concerns
- ✓ Shard keys, chunks and mongos
- ✓ Targeted versus scatter-gather

NOW Put the practices around it: model, tune, secure, back up, monitor and diagnose.



OUR PATH THROUGH MODULE 8

- 1 Model and validate documents
- 2 Query, pipeline and index habits
- 3 Authentication, roles, TLS, network
- 4 Backups, RPO, RTO and restore tests
- 5 Metrics and troubleshooting by layer
- 6 Production-readiness review

Part 1

Part 2

Part 3

Part 4

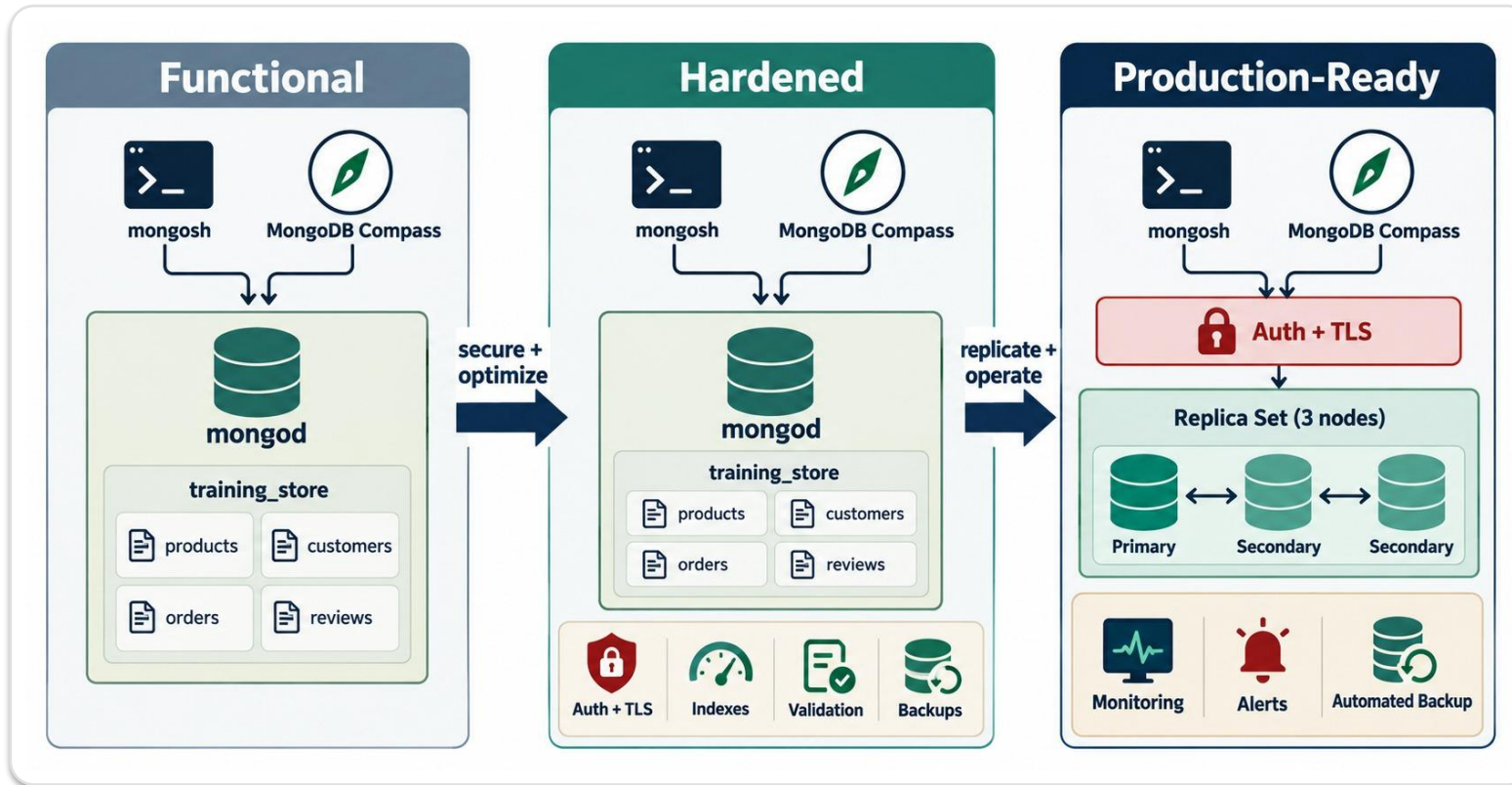
Part 5

Part 5

KEY TAKEAWAY Module 7 kept training_store available; Module 8 makes it correct, secure and recoverable.

FROM FUNCTIONAL TO PRODUCTION-READY

What changes between “it works” and “we can run it”.



Functional

One mongod, no security — where our class instance is today.

Hardened

Auth + TLS, indexes, validation and backups.

Production-ready

A replica set with monitoring, alerts and automated backup.

SAME DATA The four collections never change — the controls do.

KEY TAKEAWAY Production-ready is the same data with proven controls around it.

PART 1: DATA-MODELING GUIDELINES

Flexible schema, governed by explicit rules.



SO FAR

- ✓ You modeled training_store in Module 3
- ✓ You queried, aggregated and indexed it
- ✓ You replicated and sharded it

NEXT QUESTION

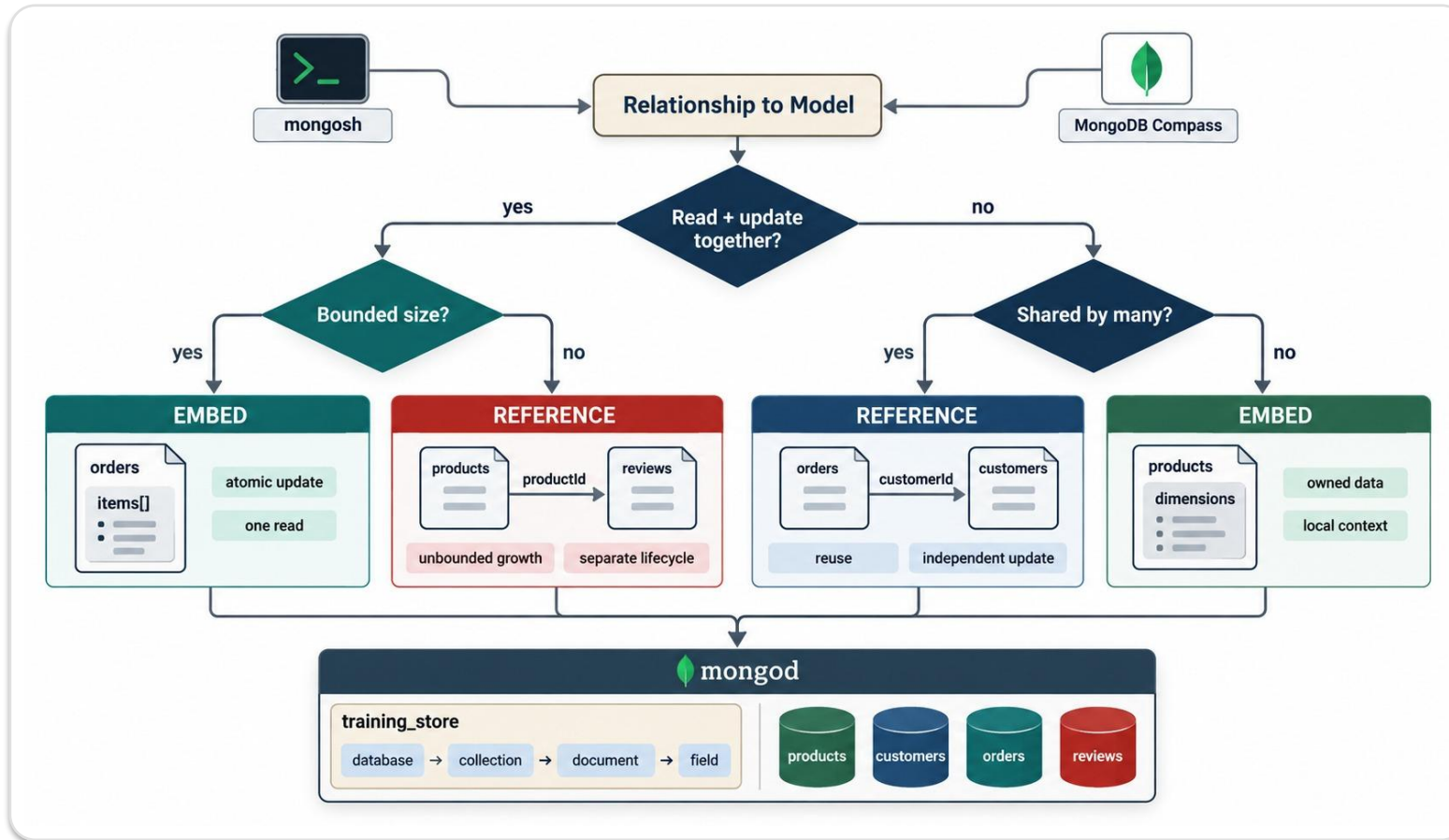
Will this document model still hold up as data, features and teams grow?

IN THIS PART

- 1 Embed or reference: document boundaries
- 2 Consistent field names and BSON types
- 3 Historical snapshots
- 4 Schema validation
- 5 Planning schema evolution

CHOOSE DOCUMENT BOUNDARIES CAREFULLY

Embed bounded, owned data — reference shared or unbounded data.



Embed

Order items and the shipping address: read with the order, bounded, owned.

Reference

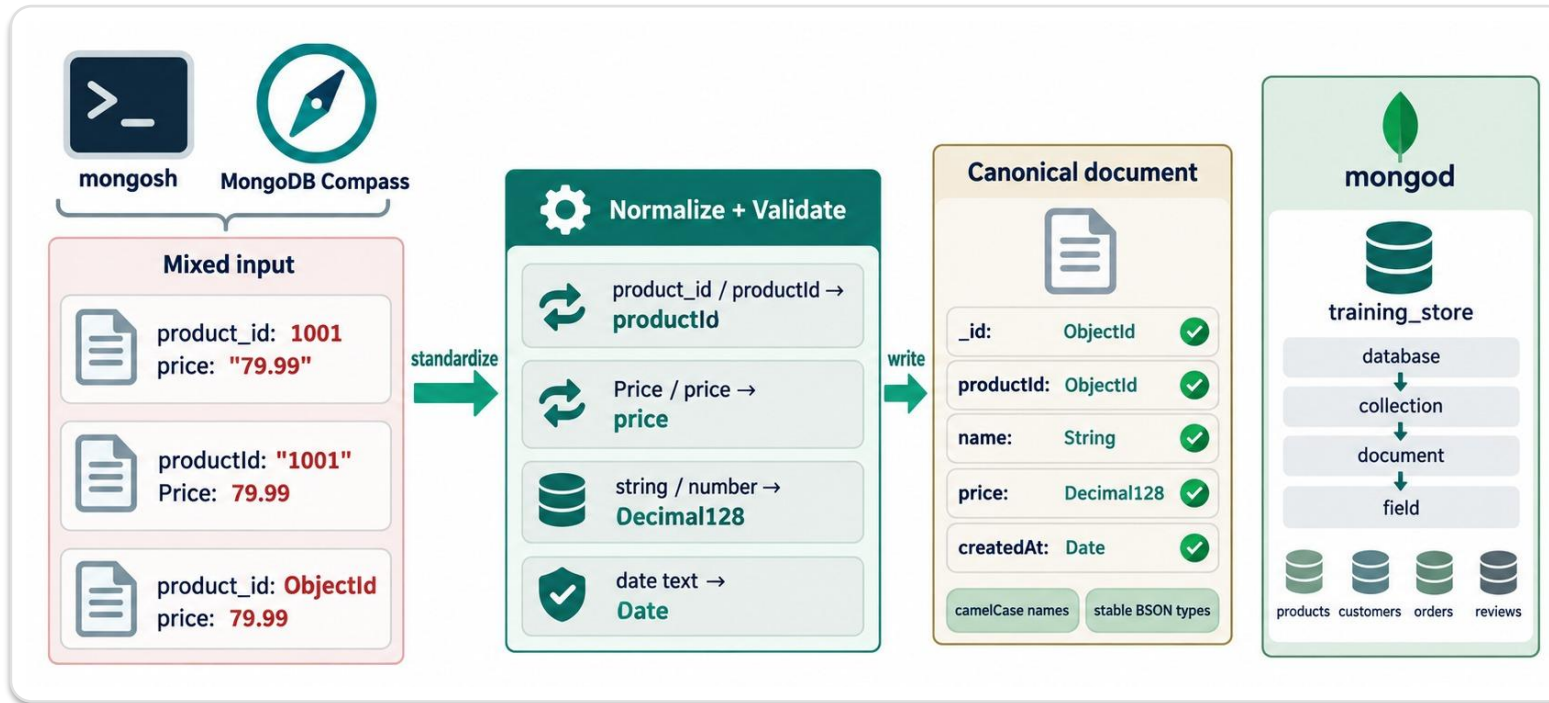
Customers are shared; reviews grow without limit.

ANTI-PATTERN Every review inside its product — an unbounded array.

KEY TAKEAWAY Embed what belongs to one parent and stays bounded; reference the rest.

USE CONSISTENT FIELD NAMES AND TYPES

One name and one BSON type per concept.



Names

camelCase, one spelling: productId — never product_id or prodId.

Types

Decimal128 for money, Date for time, ObjectId for references.

IN OUR DATA XBAD stores price as the string "49.99".

KEY TAKEAWAY A field with two spellings or two types quietly breaks queries, sorts and indexes.

FIND THE DRIFT IN TRAINING_STORE

Three queries that expose type and naming problems.

Prices stored with the wrong type

```
db.products.find(
  { price: { $type: "string" } },
  { _id: 0, sku: 1, price: 1 }
)
// { sku: 'XBAD', price: '49.99' }

db.products.countDocuments(
  { price: { $type: "decimal" } })
// 12
```

\$type

Finds values whose BSON type differs from the rule.

Fields outside the model

```
db.products.find(
  { $or: [
    { legacyName: { $exists: true } },
    { temporaryNote: { $exists: true } }
  ] },
  { _id: 0, sku: 1 }
)
// { sku: 'L190' }, { sku: 'A410' }
```

\$exists

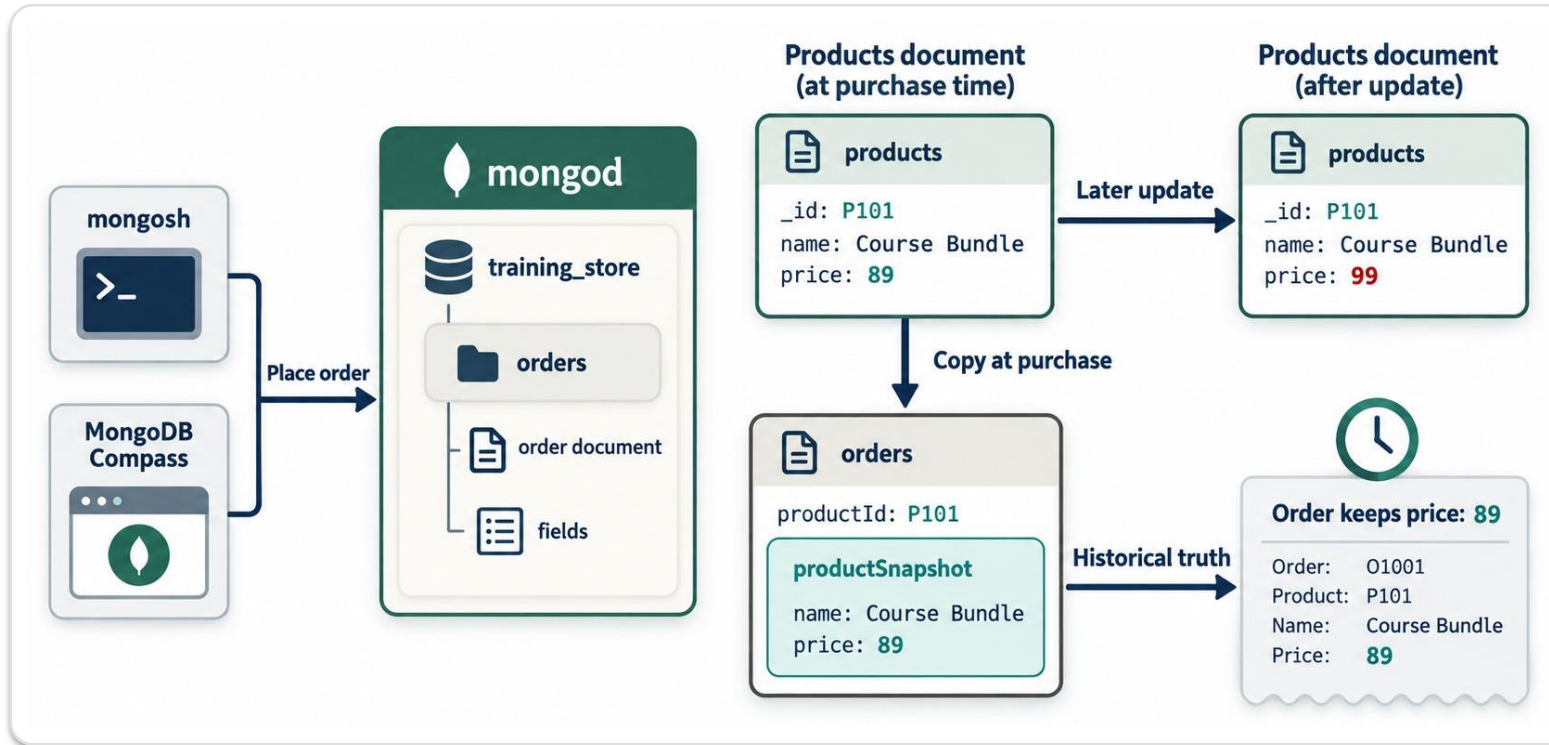
Finds stray fields left by old code or manual edits.

FIX PLAN Migrate XBAD to Decimal128, then validate.

KEY TAKEAWAY Measure the drift before you fix it — \$type and \$exists turn a hunch into a list.

PRESERVE HISTORICAL SNAPSHOTS

The order keeps what the customer actually paid.



Copy at purchase

The name and price are copied into the order.

Later update

The product's price changes; the order doesn't.

In training_store

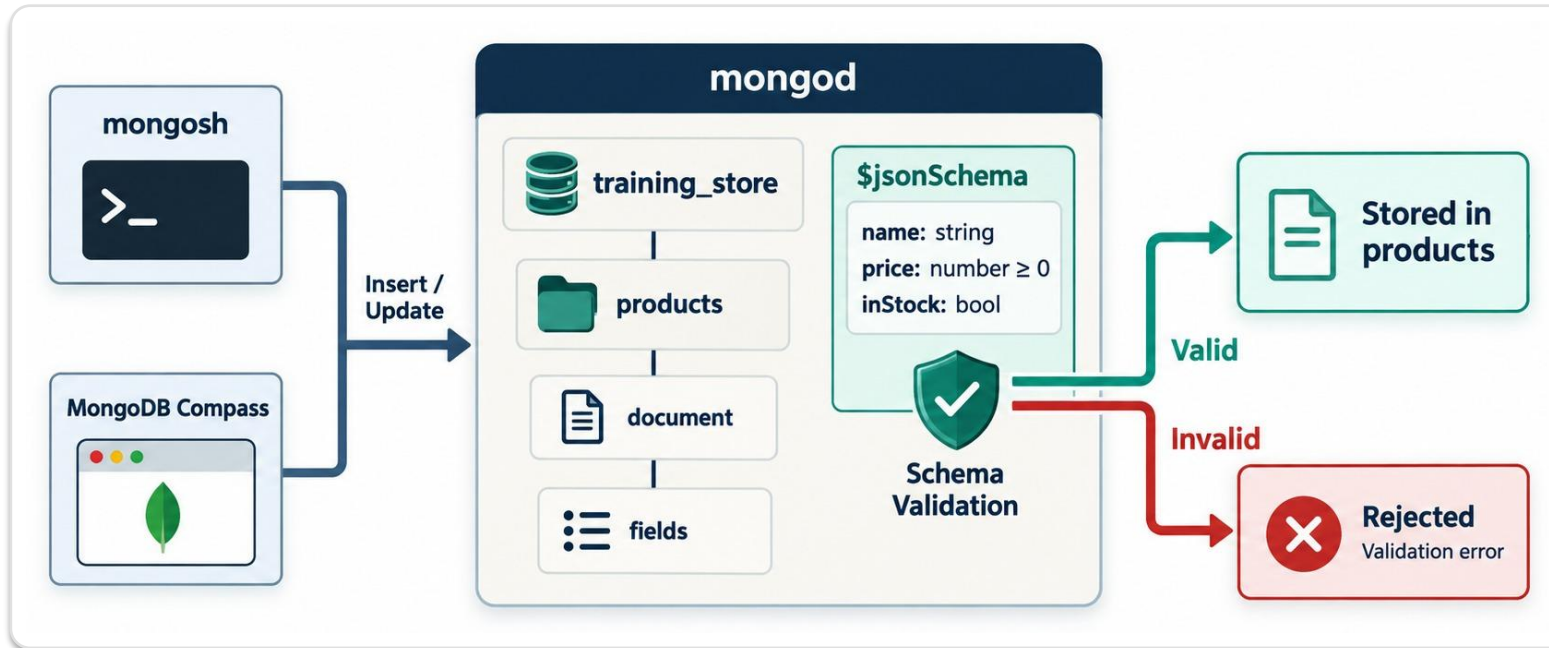
Each item keeps sku, name, quantity and unitPrice.

INTENTIONAL Snapshot duplication records history — it isn't a bug.

KEY TAKEAWAY Copy the values that must stay historically true into the order at the moment it is placed.

APPLY SCHEMA VALIDATION

The flexible model can still refuse bad data.



Required

Fields that must exist, e.g. `orderId` and `total`.

Types and enums

`total` must be decimal; `paymentStatus` one of three values.

Rejected

An invalid write fails with code 121, Document failed validation.

NOT A REPLACEMENT Keep validating in the application too.

KEY TAKEAWAY Schema validation turns the rules the model depends on into rules the database enforces.

A VALIDATOR ON A SCRATCH COLLECTION

Lab 7, Step 3 — training_store_ops, so the class orders stay untouched.

✓ Create the collection with \$jsonSchema

```
use training_store_ops
db.createCollection("orders_validated", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["orderNumber", "total",
              "paymentStatus"],
    properties: {
      orderNumber: { bsonType: "string" },
      total: { bsonType: "decimal" },
      paymentStatus: {
        enum: ["PAID", "PENDING", "FAILED"] }
    } } },
  validationAction: "error"
})
```

Prove accept and reject

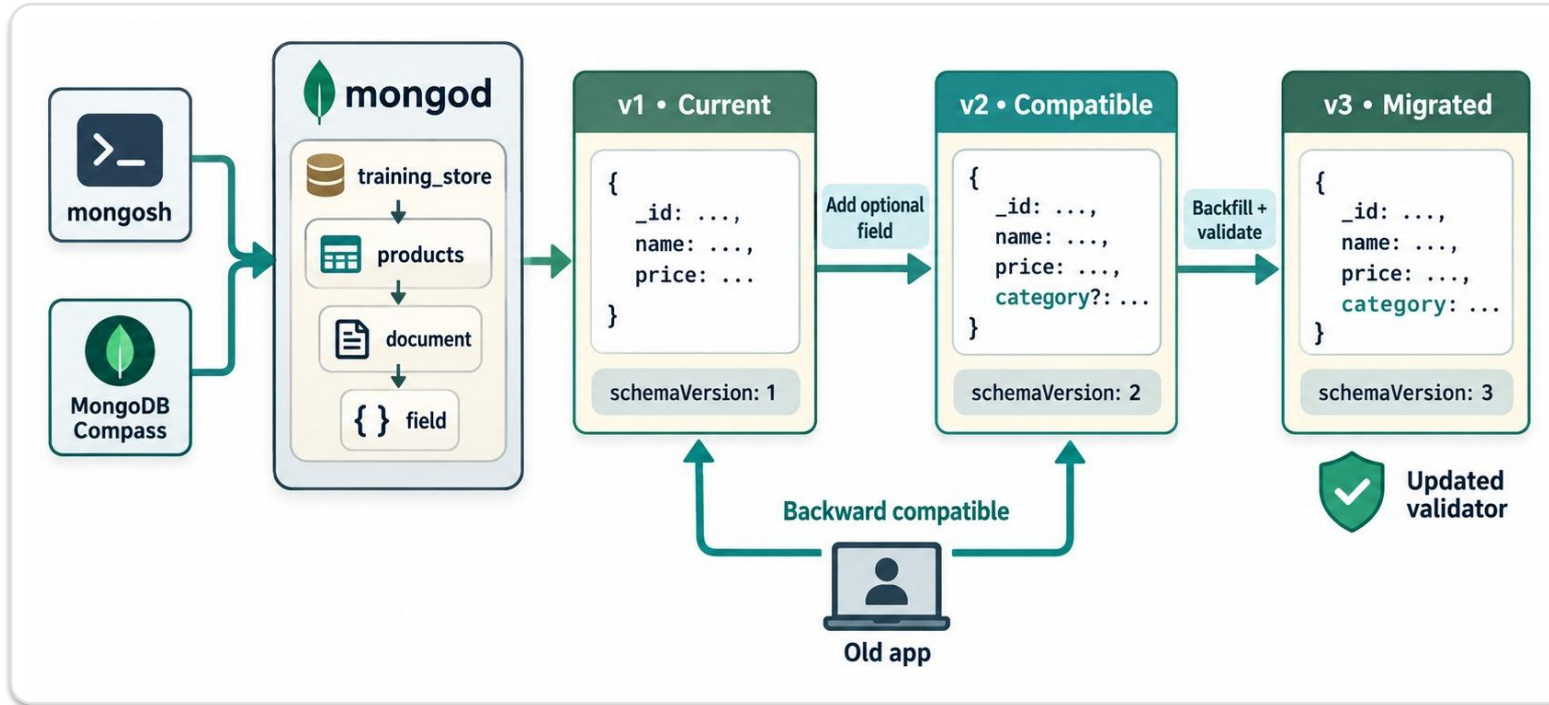
```
db.orders_validated.insertOne({
  orderNumber: "OPS-1",
  total: Decimal128("10.00"),
  paymentStatus: "PAID" })
// acknowledged: true

db.orders_validated.insertOne({
  orderNumber: "OPS-BAD",
  total: "10.00",
  paymentStatus: "PAID" })
// MongoServerError:
//   Document failed validation
```

KEY TAKEAWAY Test a validator with one good document and one bad one before you trust it.

PLAN FOR SCHEMA EVOLUTION

Change the shape in compatible steps.



MIGRATION ORDER

- ✓ Support both shapes in the app
- ✓ Start writing the new shape
- ✓ Migrate old documents in batches
- ✓ Measure progress and failures
- ✓ Tighten validation, remove old code

AVOID One huge untested updateMany on production.

KEY TAKEAWAY Evolve the schema in backward-compatible steps and tighten the validator last.

EXERCISE: PREDICT THE VALIDATOR'S VERDICT

Accepted or rejected? Decide before you run each insert.

PRACTICE EXERCISE

10 MIN

SCENARIO

orders_validated requires orderNumber (string), total (decimal) and paymentStatus (PAID, PENDING or FAILED).

```
A {orderNumber:"OPS-2",
  total:Decimal128("24.99"),
  paymentStatus:"PENDING"}
B {orderNumber:"OPS-3",
  total:Decimal128("5.00"),
  paymentStatus:"PAID_NOW"}
C {total:Decimal128("5.00"),
  paymentStatus:"PAID"}
D {orderNumber:"OPS-4",
  total:9.99, paymentStatus:"PAID"}
```

YOUR TASKS

- 1 Mark each insert accepted or rejected
- 2 Name the rule each rejection breaks
- 3 Say whether an extra field such as note is allowed
- 4 Run them on training_store_ops to check

EXPECTED OUTCOME

- ✓ A accepted
- ✓ B rejected: enum
- ✓ C rejected: required
- ✓ D rejected: 9.99 is a double
- ✓ Extra fields pass — no additionalProperties rule

KEY TAKEAWAY A validator enforces only the rules you wrote — know exactly which ones those are.

PART 2: PERFORMANCE HABITS

Observe → measure → diagnose → change → measure again.



SO FAR

- ✓ Documents are bounded and consistently typed
- ✓ Snapshots are intentional
- ✓ Validation protects the essentials

NEXT QUESTION

How do we prove a query or a report is fast — and still correct?

IN THIS PART

- 1 Filter, project, sort and limit
- 2 Reduce pipeline input early
- 3 Validate aggregation results
- 4 ESR compound indexes
- 5 Reading explain plans

ASK ONLY FOR WHAT THE PAGE NEEDS

A catalog page: filter, project, sort and limit.

✗ Anti-pattern: return everything

```
db.products.find({})  
// all 13 products, every field,  
// including attributes
```

✓ Better: one bounded, shaped query

```
db.products.find(  
  { category: "LAPTOP", active: true },  
  { _id: 0, sku: 1, name: 1, price: 1 }  
)  
.sort({ price: 1 }).limit(20)  
// L100 Business Laptop 1299.99  
// L110 Ultrabook      1899.99
```

Filter

Selective predicates; L190 is inactive and drops out.

Project

Return three fields, not attributes.

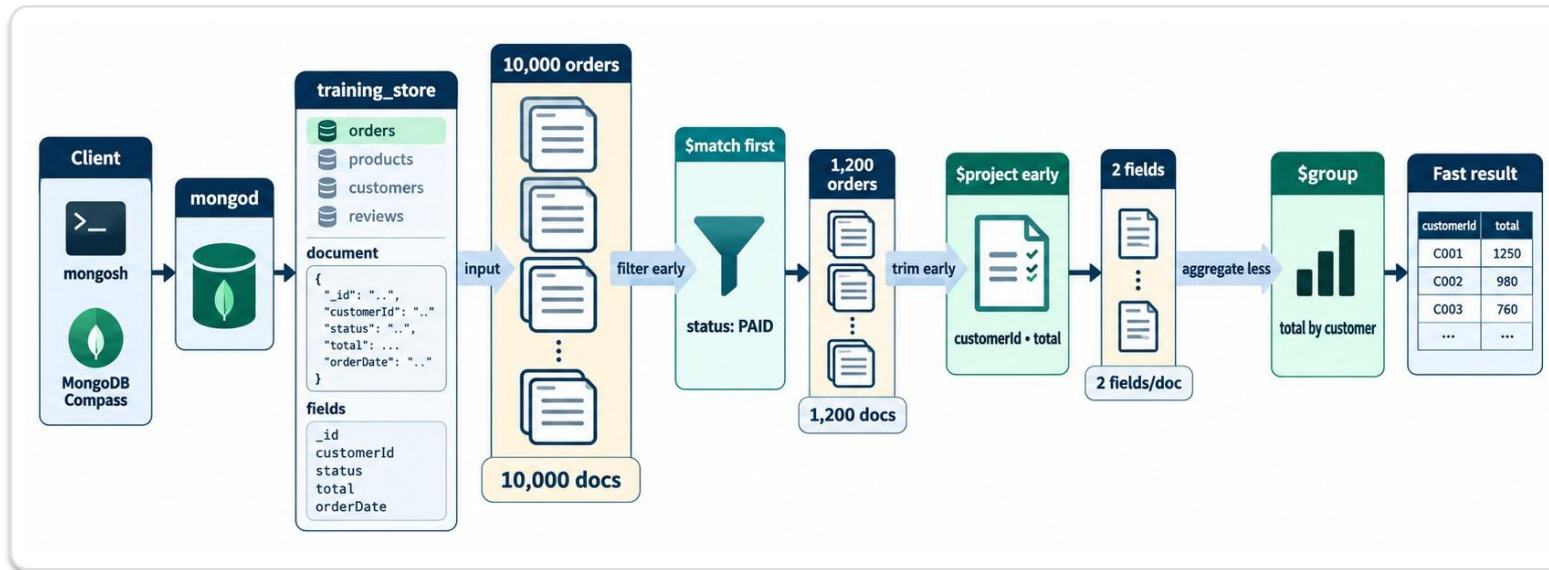
Limit

Bound every user-facing list; paginate by range.

KEY TAKEAWAY Every user-facing query should be filtered, projected, sorted and limited.

REDUCE PIPELINE INPUT EARLY

\$match and \$project before the expensive stages.



\$match first

Keep only paid orders; an early \$match can use an index.

\$project early

Carry only the fields later stages need.

In training_store

The field is paymentStatus: 13 of 17 orders are PAID.

SCALE The diagram's 10,000 orders are illustrative.

KEY TAKEAWAY Filter and trim at the start of the pipeline, so every later stage has less to do.

MONTHLY PAID REVENUE — BUILT SAFELY

Lab 7, Step 2: group whole orders, not unwound items.

✓ Pipeline

```
db.orders.aggregate([
  { $match: { paymentStatus: "PAID" } },
  { $group: {
    _id: { $dateToString: {
      format: "%Y-%m",
      date: "$createdAt" } },
    orderCount: { $sum: 1 },
    revenue: { $sum: "$total" } } },
  { $sort: { _id: 1 } }
])
```

Result

```
{ _id: '2026-07', orderCount: 3,
  revenue: Decimal128('1835.95') }
{ _id: '2026-08', orderCount: 4,
  revenue: Decimal128('2731.92') }
{ _id: '2026-09', orderCount: 6,
  revenue: Decimal128('4550.57') }
```

\$match first

Only the 13 paid orders enter the pipeline.

Group whole orders

total is summed once per order.

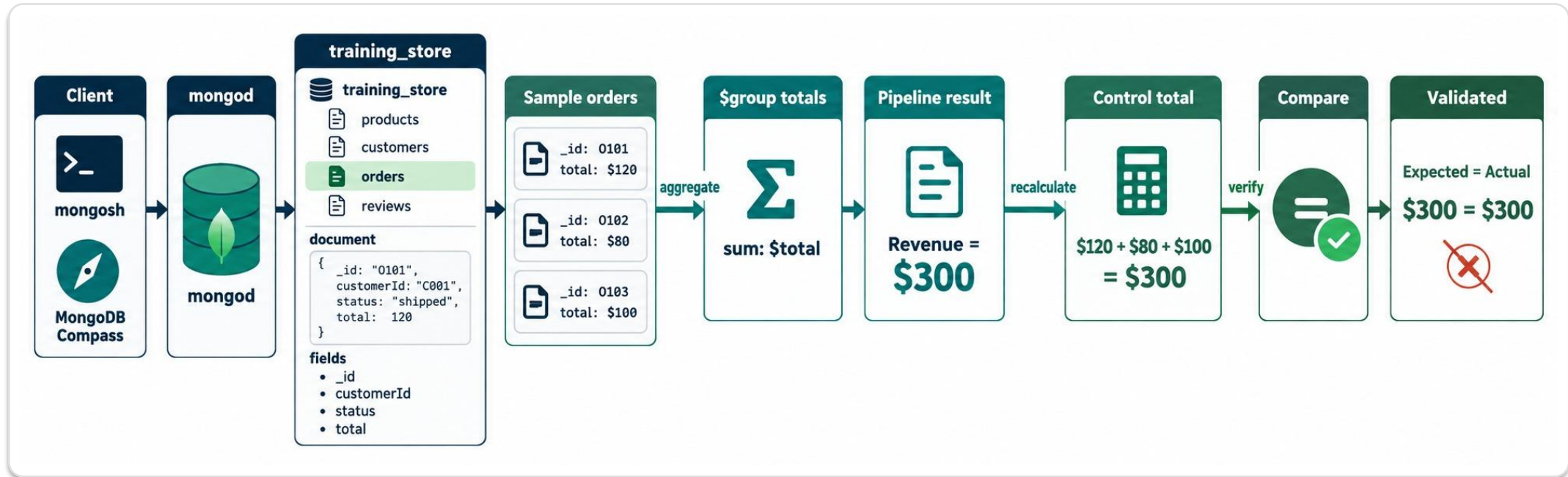
Months in UTC

\$dateToString uses UTC unless you pass a timezone.

KEY TAKEAWAY Put \$match first and sum order-level fields before any \$unwind.

VALIDATE AGGREGATION CALCULATIONS

A fast report can still be wrong.



Control total

`countDocuments({ paymentStatus: "PAID" })` → 13 orders.

The \$unwind trap

Unwind items, then sum `$total`: 13 orders become 21 rows.

Our numbers

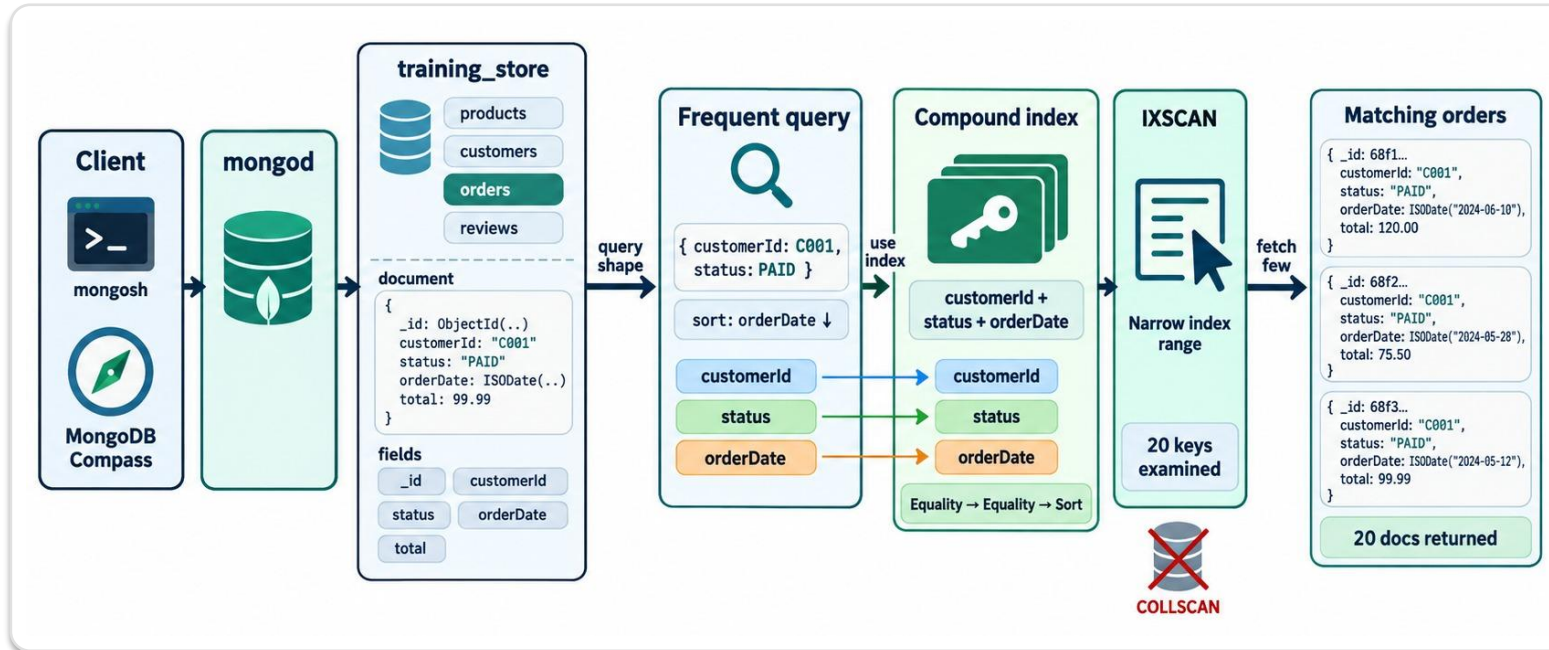
True revenue 9118.44 — the unwound sum says 12119.25.

RULE Check every report against an independent control.

KEY TAKEAWAY Verify a pipeline's totals against an independent control before anyone trusts the report.

INDEXING BEST PRACTICES

Equality, then Sort, then Range — built from the real query shape.



The same shape in training_store

```
db.orders.createIndex({
  customerId: 1,
  paymentStatus: 1,
  createdAt: -1 })
```

ESR

Equality fields first, the sort next, range fields last.

Each index costs

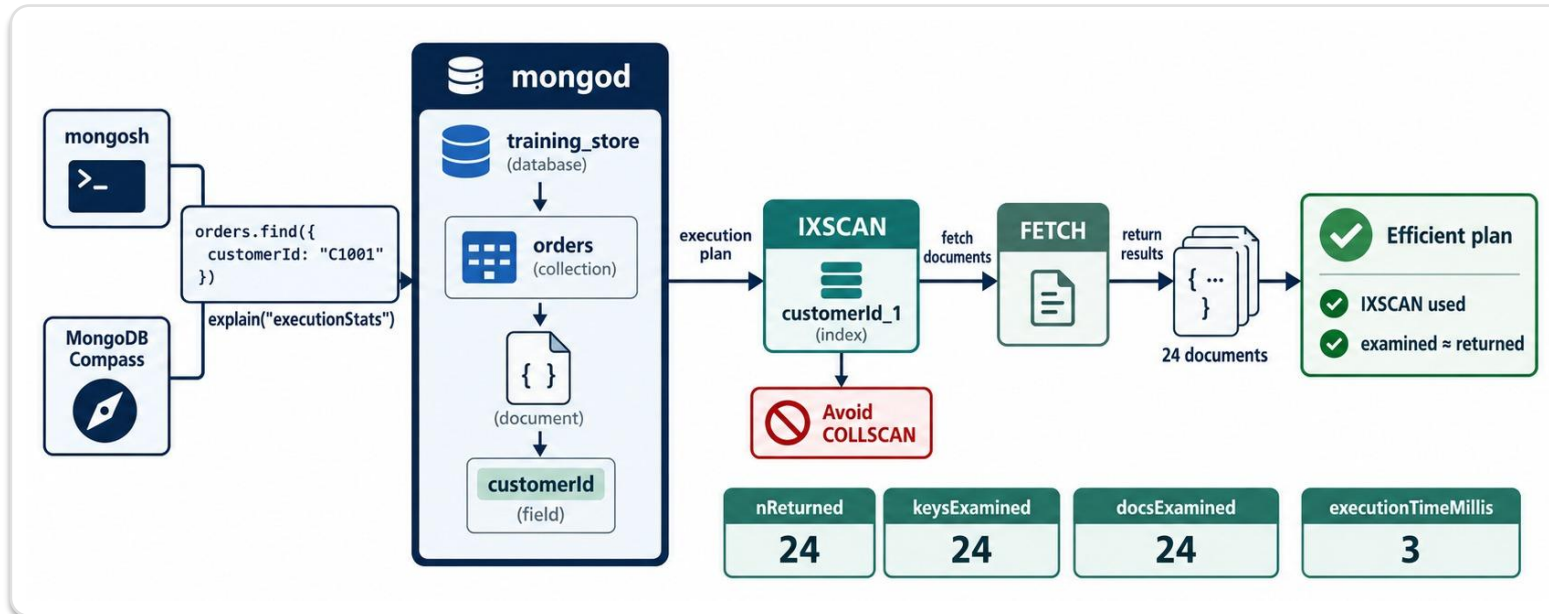
Disk, cache and extra work on every write.

BEFORE DROPPING Hide the index, monitor, then drop.

KEY TAKEAWAY Build each compound index from a real query shape, in ESR order, and know what it costs.

REVIEW EXPLAIN PLANS

explain("executionStats") is the evidence behind every index.



C101's paid history

```
db.orders.find({
  customerId: c101,
  paymentStatus: "PAID" })
.sort({ createdAt: -1 })
.explain("executionStats")
```

No index

COLLSCAN: 17 documents examined, 4 returned, plus a SORT.

ESR index

IXSCAN: 4 keys, 4 documents, no SORT stage.

KEY TAKEAWAY Compare examined with returned — a large gap means a missing or unsuitable index.

EXERCISE: DESIGN THE INDEX FOR EACH SHAPE

Three real training_store queries — one compound index each.

PRACTICE EXERCISE

15 MIN

SCENARIO

Write the index for each shape, label each key E, S or R, then confirm with explain.

- 1 Catalog: category "LAPTOP", active true, sort price asc
- 2 History: customerId = c101, sort createdAt desc
- 3 Report: paymentStatus "PAID", createdAt in September

YOUR TASKS

- 1 Write an index for each shape
- 2 Label every key E, S or R
- 3 Check the plan with explain()
- 4 Name the extra cost of each index

EXPECTED OUTCOME

- ✓ 1 { category, active, price }
- ✓ 2 { customerId: 1, createdAt: -1 }
- ✓ 3 { paymentStatus, createdAt }
- ✓ IXSCAN and no SORT stage
- ✓ Each slows inserts and updates

KEY TAKEAWAY An index is designed for a query shape — write the shape down first.

PART 3: SECURITY

Authentication, authorization, TLS and network exposure.



SO FAR

- ✓ The model is bounded and validated
- ✓ Queries are shaped and indexed
- ✓ Results are checked for correctness

NEXT QUESTION

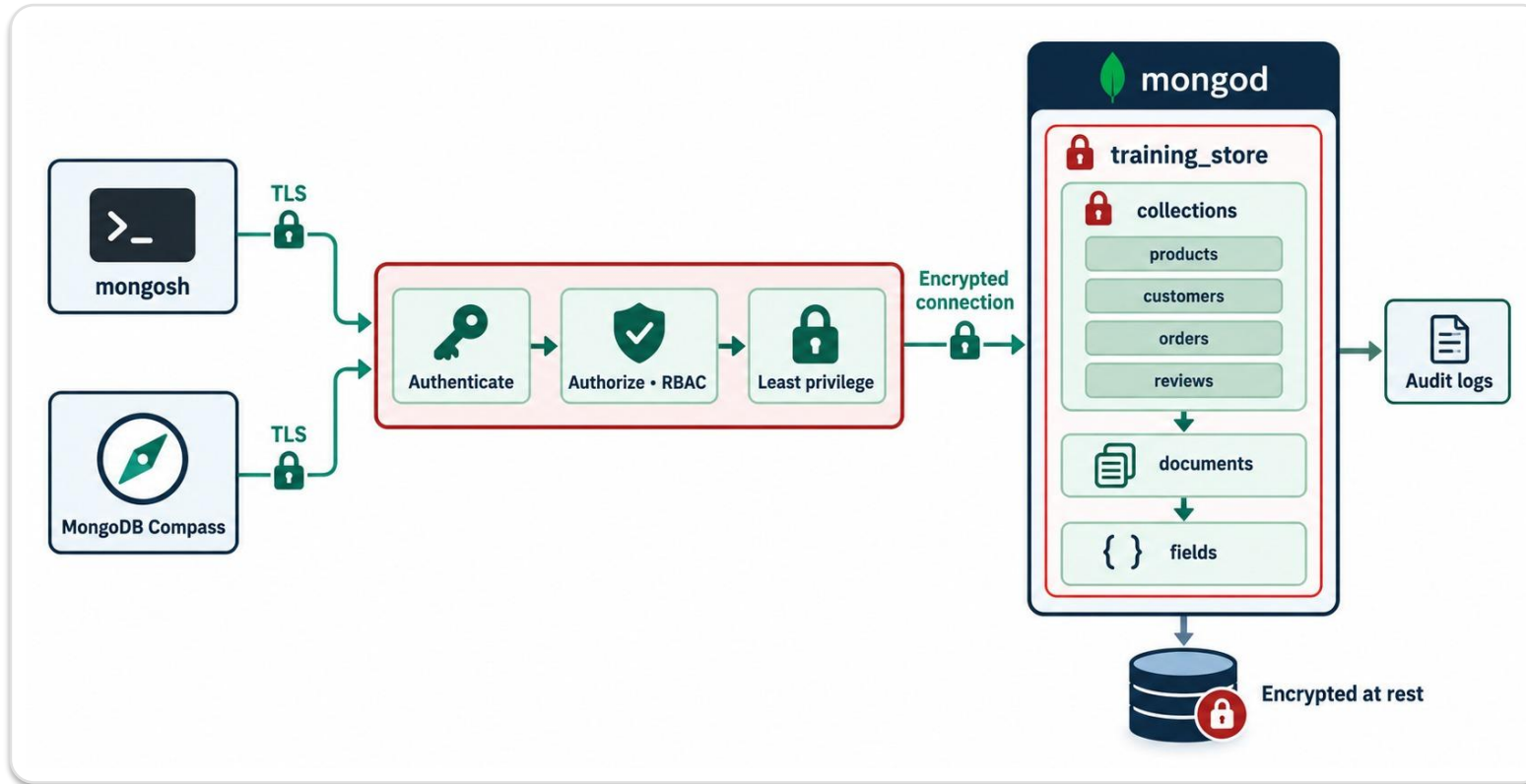
Who can reach the database, who are they, and what may they do?

IN THIS PART

- 1 Security in layers
- 2 Authentication
- 3 Least-privilege roles
- 4 Network exposure and TLS
- 5 Configuration and credentials

MONGODB SECURITY PRINCIPLES

Every request passes several controls.



In transit

TLS from every client and between members.

Identity and rights

Authenticate, authorize with roles, least privilege.

At rest and after

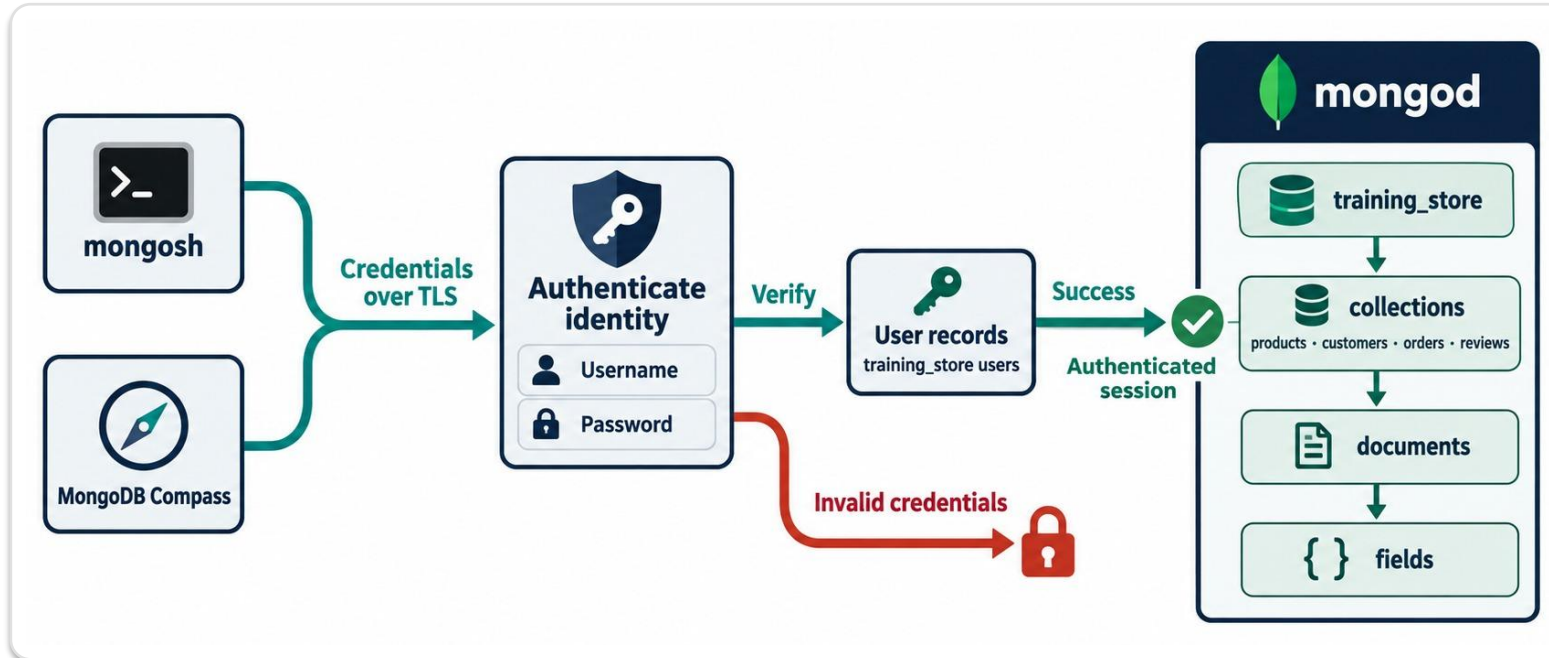
Encrypted storage; audit logs of security events.

DEFENSE IN DEPTH If one layer fails, the others still hold.

KEY TAKEAWAY Layer the controls, so one mistake never exposes everything.

AUTHENTICATION

Who is connecting?



Turn on access control

```
# mongod.conf
security:
  authorization: enabled
```

First user first

Create an administrator before enforcing it.

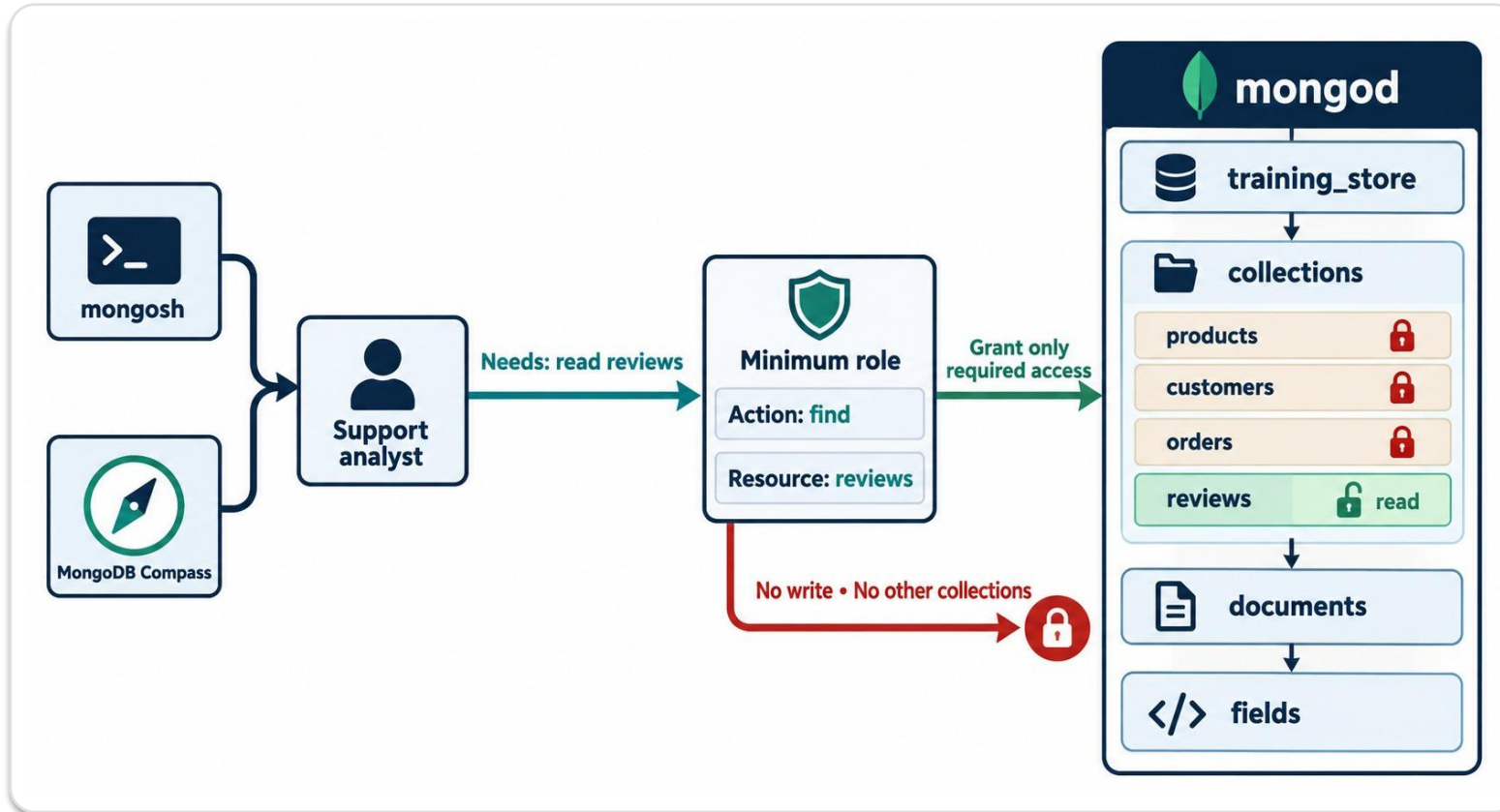
Identity, not location

A trusted network is not proof of identity.

KEY TAKEAWAY Enable access control, so every connection must prove who it is.

PRINCIPLE OF LEAST PRIVILEGE

Grant only the access each job requires.



Application

readWrite on training_store — never clusterAdmin.

Reporting

read only.

Support

find on reviews, as in the diagram.

Operations

Monitoring, backup and DBA each get their own identity.

KEY TAKEAWAY Give every identity its own account and only the privileges its job needs.

USERS AND ROLES — SKETCHED, NOT RUN

Placeholder names, prompted passwords; a disposable instance only.

✓ Application and reporting users

```
use training_store
db.createUser({
  user: "trainingStoreApp",
  pwd: passwordPrompt(),
  roles: [{ role: "readWrite",
            db: "training_store" }]
})
db.createUser({
  user: "reportUser",
  pwd: passwordPrompt(),
  roles: [{ role: "read",
            db: "training_store" }]
})
```

passwordPrompt()

The password is typed, never stored in the script or history.

✓ A custom role: read reviews only

```
db.createRole({
  role: "reviewReader",
  privileges: [{
    resource: { db: "training_store",
                collection: "reviews" },
    actions: ["find"]
  }],
  roles: []
})
```

Built-in roles

readWrite and read cover most services.

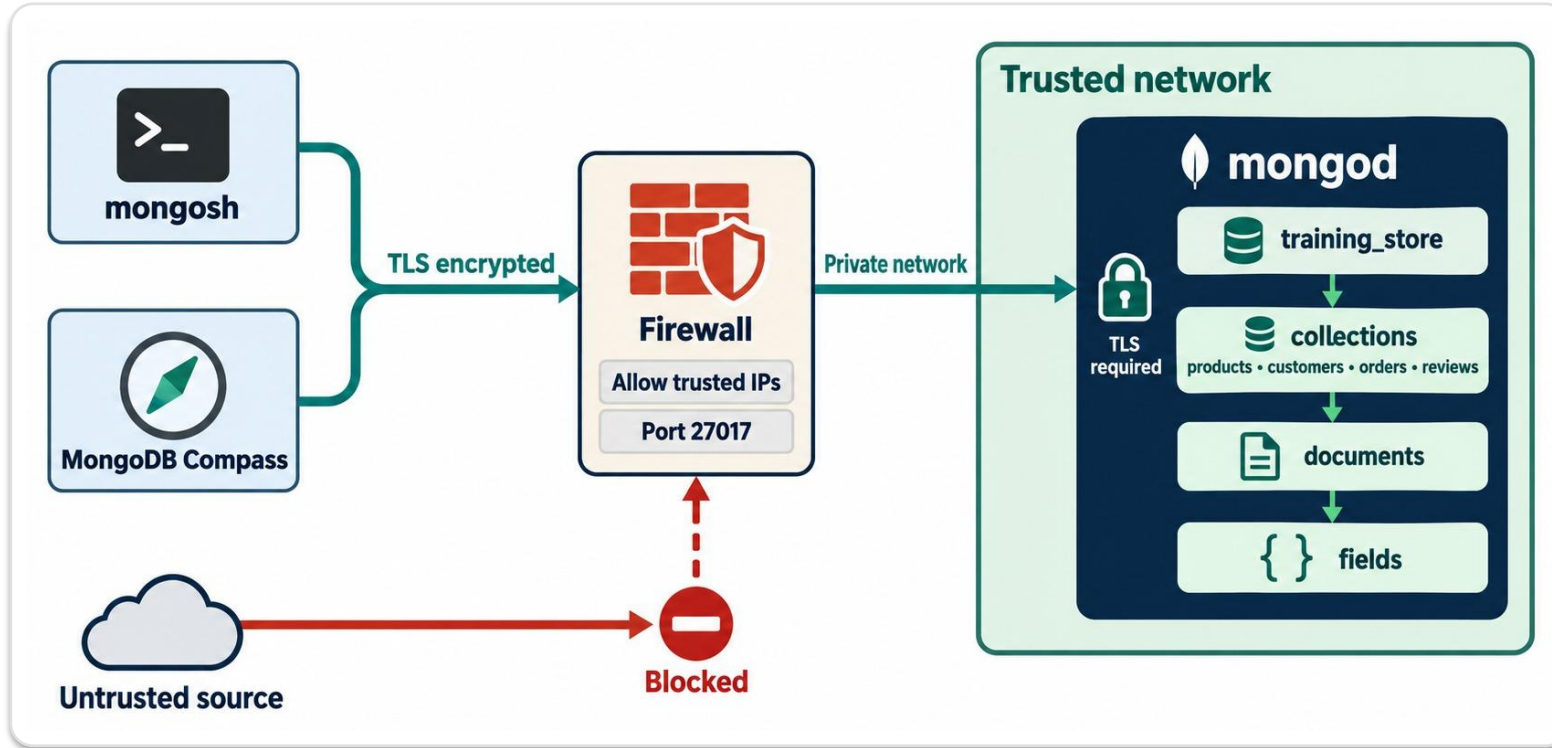
Custom roles

Narrow a role to single collections and actions.

KEY TAKEAWAY Create one identity per job, with the smallest role that works, and never type a real password into a script.

NETWORK SECURITY

Make the database unreachable to anyone who shouldn't reach it.



Firewall

Allow trusted IPs on 27017; block everything else.

Private network

mongod lives on a private subnet, never the public internet.

bindIp

Listen only on localhost and the private address clients use.

ATLAS IP access lists and private endpoints.

KEY TAKEAWAY Don't expose port 27017 broadly — restrict who can reach mongod before anyone tries to log in.

SECURE CONFIGURATION AND CONNECTION STRINGS

A hardened mongod.conf — and the anti-pattern it replaces.

✓ Hardened (sketch)

```
net:
  port: 27017
  bindIp: 127.0.0.1,10.0.1.15
  tls:
    mode: requireTLS
    certificateKeyFile: /etc/mongo/server.pem
    CAFile: /etc/mongo/ca.pem
security:
  authorization: enabled
```

✗ Anti-pattern — never ship this

```
net:
  bindIp: 0.0.0.0 # every interface
# no security section:
#   access control off, no TLS

// URI with a password in source code
mongodb://admin:<password>@db:27017/
```

Listen narrowly

Localhost plus one private address.

TLS everywhere

Validate certificates; never disable checks.

Secrets

Load the URI from a secret store at runtime.

KEY TAKEAWAY Bind narrowly, require TLS and authentication, and keep credentials out of code and configuration files.

SECURITY MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs that leave real databases exposed.

MISCONCEPTION

REALITY

ANTI-PATTERNS TO AVOID

It's on our network, so it's safe

Authenticate every connection — location isn't identity

✗ Port 27017 open to the internet

The app needs admin to be safe

Least privilege: readWrite on its own database

✗ One shared administrator account

TLS is only for the internet

Encrypt inside the data center and between members too

✗ Passwords in code, Git or screenshots

Encryption replaces password hashing

Hash application passwords; encrypt sensitive fields

✗ Unencrypted backup files

Fix not-authorized by granting root

Grant the one missing privilege

✗ Production data copied into development

REMEMBER Unreachable, authenticated, least privilege, encrypted.

KEY TAKEAWAY Most breaches come from simple gaps — exposure, shared accounts and leaked credentials.

PART 4: BACKUPS AND RECOVERY

Replication keeps you running. Backups let you go back.



SO FAR

- ✓ Access is authenticated and least-privilege
- ✓ Traffic is encrypted; the network is closed
- ✓ Credentials live outside the code

NEXT QUESTION

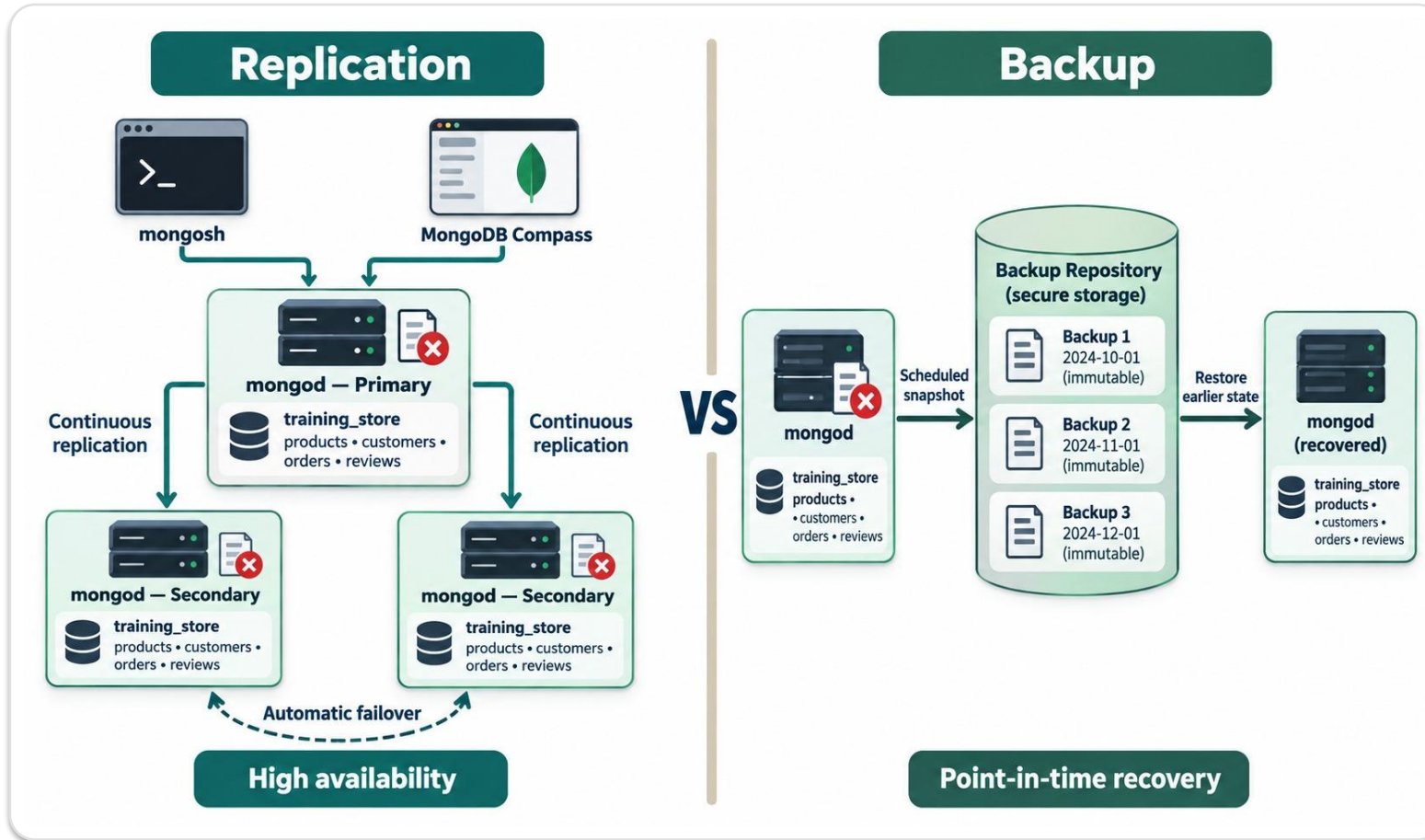
If someone deletes the orders tonight, what do we lose — and how long until we are back?

IN THIS PART

- 1 Replication versus backup
- 2 Recovery objectives: RPO and RTO
- 3 Restore testing

REPLICATION VS. BACKUP

Two different capabilities — you need both.



An accidental delete

```
db.orders.deleteMany({})  
// replicated to every member
```

Replication

Copies every change, good or bad, in seconds.

Backup

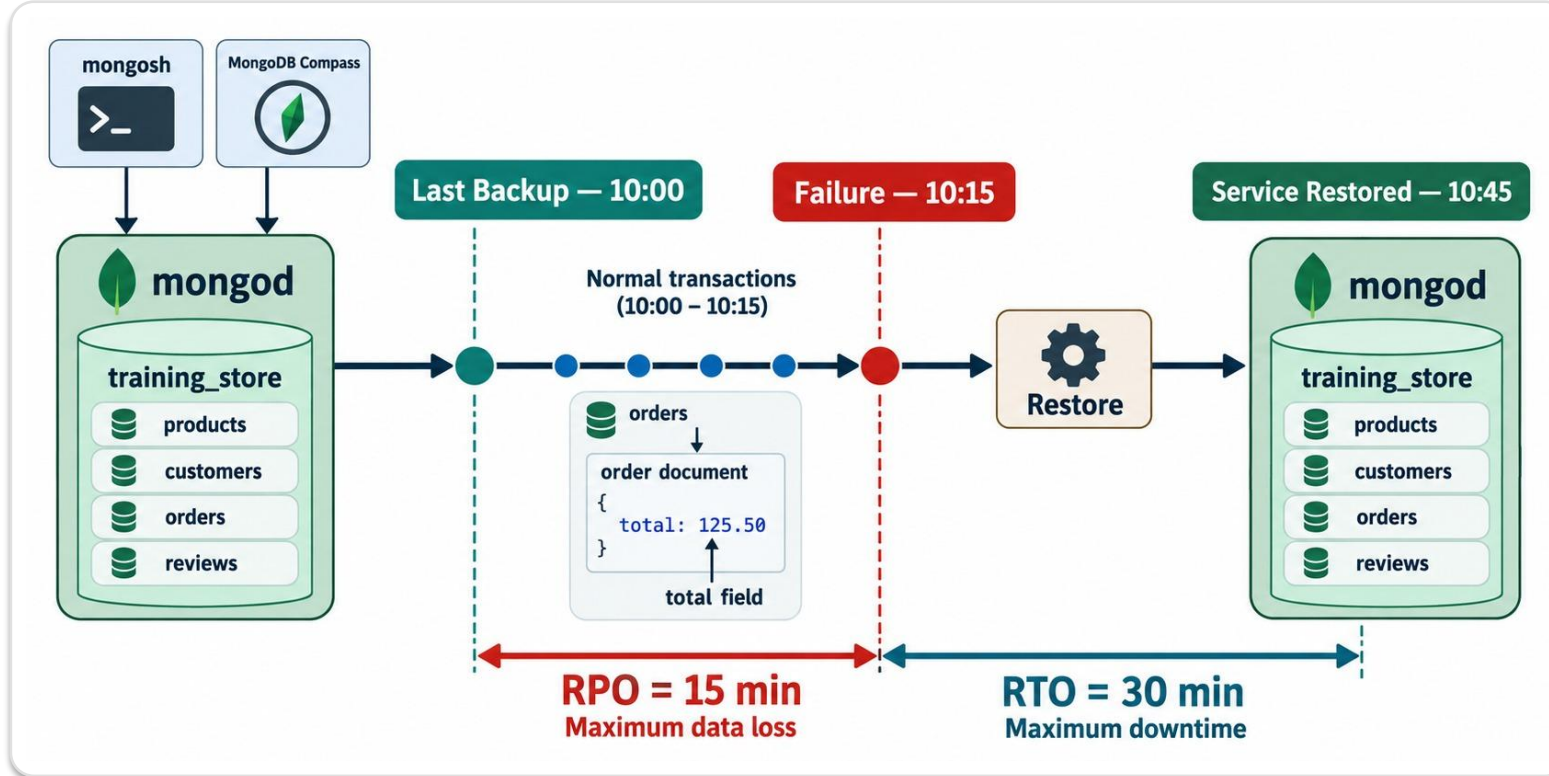
Immutable copies from before the mistake.

RULE Replication is not a backup.

KEY TAKEAWAY Replication protects availability; only an independent backup protects you from a bad change.

RECOVERY POINT AND RECOVERY TIME OBJECTIVES

How much data you may lose, and how long you may be down.



RPO = 15 min

Changes since the 10:00 backup are at risk.

RTO = 30 min

Service restored at 10:45 after a 10:15 failure.

Set per data set

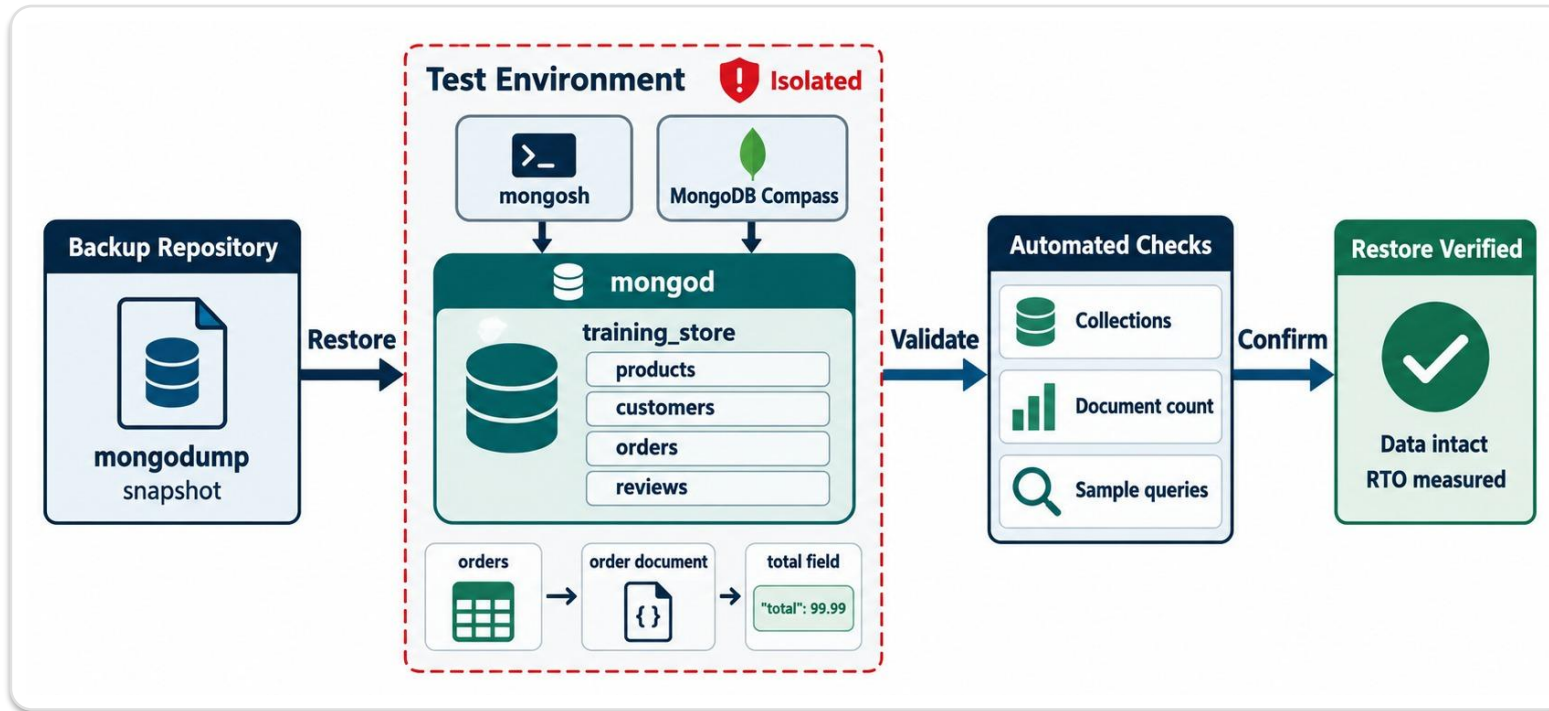
Orders and payments tight; reviews more tolerant.

EXERCISE 8.3 You'll set these for training_store.

KEY TAKEAWAY RPO and RTO are business targets in minutes — choose the backup method that meets them.

RESTORE TESTING

A backup is proven only by a restore.



Lab 7, Step 4

```
mongodump --uri="mongodb://localhost:27017"
--db=training_store --out=<dir>
mongorestore --
uri="mongodb://localhost:27017"
--db=training_store_restore --drop
<dir>/training_store
```

Verify

17 orders, 13 products, a sample order and its indexes.

NEVER Restore over training_store itself.

KEY TAKEAWAY Schedule restore tests into an isolated target, and measure the time against the RTO.

PART 5: OPERATE AND DIAGNOSE

Monitor the right signals and troubleshoot from evidence.



SO FAR

- ✓ Replication is not a backup
- ✓ RPO and RTO are time targets
- ✓ Restores are tested, not assumed

NEXT QUESTION

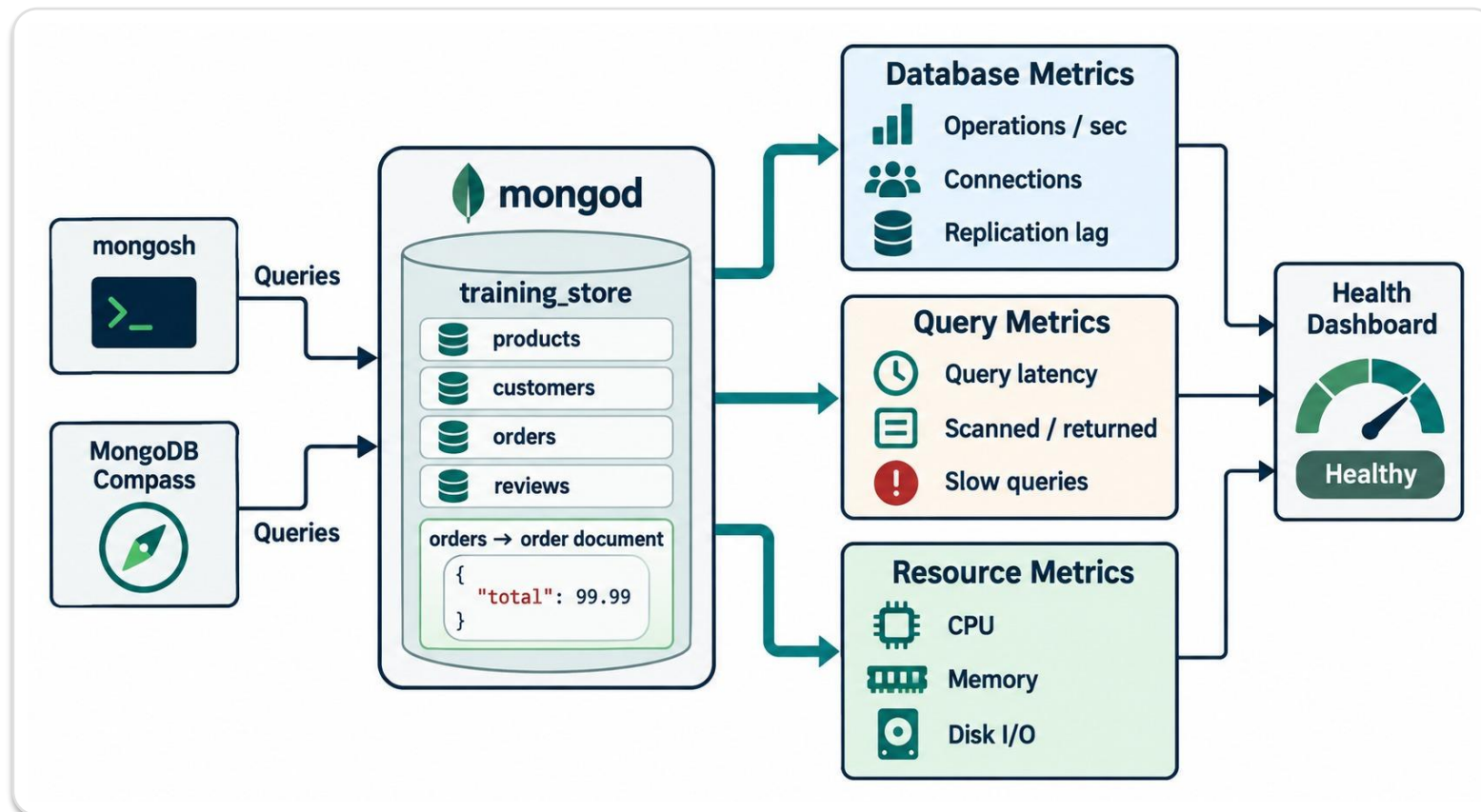
Something is wrong in production — how do we find out what, and fix only that?

IN THIS PART

- 1 Metrics that matter
- 2 A troubleshooting method
- 3 Common errors by layer
- 4 A slow-search investigation
- 5 Safe error handling
- 6 The production-readiness checklist

DATABASE, QUERY, AND RESOURCE METRICS

Three groups of signals — one health view.



Sample slow operations

```
db.setProfilingLevel(1,
  { slows: 200, sampleRate: 0.2 })
db.system.profile.find()
  .sort({ ts: -1 }).limit(5)
db.setProfilingLevel(0)
```

Also watch

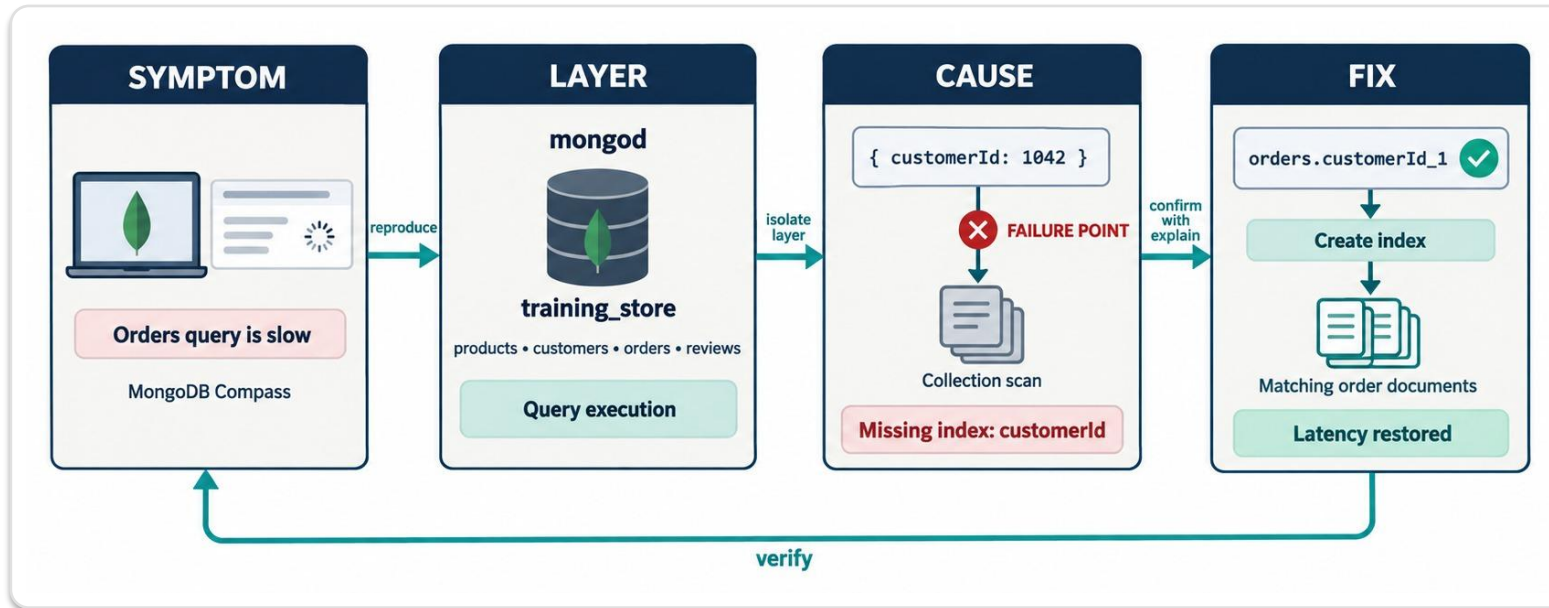
Disk space, oplog window, backup status, authentication failures.

PROFILER Adds overhead — enable it deliberately.

KEY TAKEAWAY Monitor query evidence and replication, not only CPU — slow queries show up there first.

TROUBLESHOOTING METHODOLOGY

Symptom → scope → evidence → layer → cause → fix → verify.



Precise symptom

Not "MongoDB is slow" but "order history went from 100 ms to 2.5 s".

Scope and evidence

One query or all? Logs, explain, metrics, recent changes.

Smallest safe fix

One index, one role, one setting — then verify.

LOOP Re-run the same test after every fix.

KEY TAKEAWAY Troubleshoot from evidence, one layer at a time, and verify with the same test.

COMMON PROBLEMS, BY LAYER

The error message usually names the layer.

Symptom	Likely layer	First check
Connection refused	Process, network	mongod up? host, port, firewall
Authentication failed	Security	user, password, authSource
not authorized	Security: roles	user's roles on that collection
E11000 duplicate key	Index	which unique index
Document failed validation	Schema	missing field, type, enum
Slow query	Query, index	explain: COLLSCAN, SORT
Lag or no primary	Replication	rs.status(), majority
Every shard contacted	Sharding	shard key in the filter?

Authenticated ≠ authorized

Wrong authSource fails login; a missing role fails the command.

Never

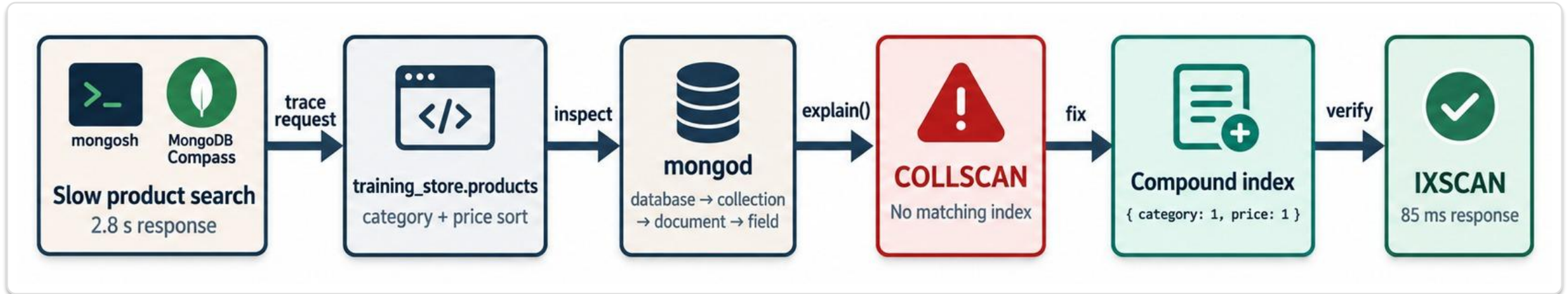
Grant root, disable auth or bind to every interface to make an error go away.

EXERCISE Name the layer for five real messages.

KEY TAKEAWAY Read the error, name the layer, then check that layer first.

SLOW APPLICATION INVESTIGATION

Follow the user's request down to the plan.



Trace

The slow search filters by category and sorts by price.

Explain

COLLSCAN — no index matches the shape.

Fix and verify

{ category: 1, price: 1 } → IXSCAN, 85 ms.

SAME INDEX The one Module 1 showed in mongosh.

KEY TAKEAWAY Start from the request the user felt, and verify with the same request.

HANDLE ERRORS SAFELY

Business messages, repeatable writes and bounded waits.

Expected conflict → safe message

```
db.customers.insertOne({
  customerNumber: "C999",
  contact: { email: "aisha@example.com" }
})
// MongoServerError: E11000 duplicate
//   key error ... index: email_unique
// user sees: "An account already
//   exists for that email."
```

Log internally

The raw error, securely, with a request ID.

Idempotent

A retry can't ship the order twice.

Timeouts

No operation waits forever.

✓ Guarded, repeatable transition

```
db.orders.updateOne(
  { orderNumber: "06302",
    fulfillmentStatus: "PROCESSING" },
  { $set: { fulfillmentStatus: "SHIPPED" },
    $currentDate: { shippedAt: true } }
)
// 1st run: modifiedCount 1
// 2nd run: matchedCount 0

db.orders.find({ fulfillmentStatus:
  "PROCESSING" }).maxTimeMS(3000)
```

KEY TAKEAWAY Turn database errors into safe messages, and design writes that are safe to retry.

EXERCISE: WHICH LAYER IS IT?

Five messages from the training_store application.

PRACTICE EXERCISE

10 MIN

SCENARIO

Name the layer and the first check for each message.

```
A connect ECONNREFUSED
  127.0.0.1:27017
B Authentication failed
  (user trainingStoreApp)
C not authorized on training_store
  to execute command { find:
    "orders" }
D E11000 ... index: email_unique
E Document failed validation
```

YOUR TASKS

- 1 Name the layer for A–E
- 2 Write the first check or command
- 3 Say what you would not do
- 4 Pick one to add to a runbook

EXPECTED OUTCOME

- ✓ A process or network: is mongod up?
- ✓ B identity: authSource and user
- ✓ C roles: grant only find on orders
- ✓ D business conflict: safe message
- ✓ E schema: which rule failed

KEY TAKEAWAY The message names the layer — read it before you change anything.

PRODUCTION-READINESS CHECKLIST

Six areas — each needs evidence, not assumptions.

DATA MODEL

- ✓ Arrays bounded; snapshots intentional
- ✓ One name and one BSON type per field
- ✓ Validator plus a migration plan

QUERIES AND AGGREGATION

- ✓ Filtered, projected, limited
- ✓ \$match first; no \$unwind double count
- ✓ Results checked against a control

INDEXES

- ✓ Each index serves a named shape
- ✓ ESR order confirmed with explain
- ✓ Redundant keys hidden, then dropped

SECURITY

- ✓ Authentication on; least privilege
- ✓ TLS; private network; no public 27017
- ✓ Secrets in a manager, not in code

AVAILABILITY AND RECOVERY

- ✓ Replica set; failover tested
- ✓ Backups separate from replication
- ✓ Restore tested against RPO and RTO

OPERATIONS

- ✓ Metrics, slow queries, alerts
- ✓ Runbooks with owners
- ✓ Changes reviewed with explain

KEY TAKEAWAY Go live only when every area has evidence — a single “not ready” area blocks the launch.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 Why design documents around access patterns?

2 When should related data be referenced?

3 What does schema validation add?

4 Why can \$unwind over-count revenue?

5 What does the ESR rule say?

6 Examined >> returned — what does it suggest?

7 Why is replication not a backup?

8 What do RPO and RTO measure?

9 Authentication vs. authorization?

10 How do you start on a slow endpoint?

EXERCISE 8.1 — REVIEW A QUERY AND PROJECTION

Module 8 checkpoint A: from `find({})` to a bounded, indexed query.

OFFICIAL CHECKPOINT A

15 MIN

labs/day-03/exercises/exercise-8.1-review-a-query-and-projection.md

SCENARIO

The app needs C101's paid orders, newest first, ten at a time: `orderId`, `fulfillmentStatus`, `total` and `createdAt`. Today it runs:

```
db.orders.find({})
```

YOUR TASKS

- 1 Add a selective filter
- 2 Project the four fields
- 3 Sort newest first and limit to ten
- 4 Name a compound index, labelled E or S

SOLUTION (AFTER DEBRIEF)

- ✓ `customerId: c101, paymentStatus: "PAID"`
- ✓ `{ _id: 0, orderId: 1, ... }`
- ✓ `.sort({ createdAt: -1 }).limit(10)`
- ✓ `{ customerId: 1, paymentStatus: 1, createdAt: -1 }`
- ✓ 4 orders: O6306, O6301, O6201, O6101

KEY TAKEAWAY A production query is selective, shaped, ordered, bounded — and backed by an index.

EXERCISE 8.2 — DESIGN LEAST-PRIVILEGE ROLES

Module 8 checkpoint B: a permit / deny matrix for six identities.

OFFICIAL CHECKPOINT B

20 MIN

labs/day-03/exercises/exercise-8.2-design-least-privilege-roles.md

SCENARIO

For each identity, list one permitted and one prohibited operation on training_store.

- 1 Application service
- 2 Reporting service
- 3 Support engineer
- 4 Monitoring service
- 5 Backup service
- 6 Database administrator

YOUR TASKS

- 1 Fill the permit / deny matrix
- 2 Choose a role for each identity
- 3 Explain why the app never uses clusterAdmin
- 4 Use placeholder names only

SOLUTION (AFTER DEBRIEF)

- ✓ App: readWrite, never userAdmin
- ✓ Reporting: read only
- ✓ Support: limited read, never drop
- ✓ Monitoring: clusterMonitor, no writes
- ✓ Backup: backup role; DBA: separate identity

KEY TAKEAWAY Least privilege limits the blast radius of any stolen credential.

EXERCISE 8.3 — DEFINE RPO AND RTO

Module 8 checkpoint C: recovery targets for each part of the store.

OFFICIAL CHECKPOINT C

15 MIN

labs/day-03/exercises/exercise-8.3-define-rpo-and-rto.md

SCENARIO

Treat training_store as a small retailer, not a global bank.

Data sets to plan:
product catalog
customer profiles
orders
payments
reviews

YOUR TASKS

- 1 Write RPO and RTO for each data set
- 2 Pick a backup frequency that meets each RPO
- 3 Describe how you would prove an orders restore

SOLUTION (AFTER DEBRIEF)

- ✓ Orders, payments: RPO in minutes
- ✓ Catalog: longer RPO is acceptable
- ✓ Reviews: the most tolerant
- ✓ Checkout RTO shorter than reviews
- ✓ Prove it: counts, O5001, indexes, a read

KEY TAKEAWAY A backup job that succeeded is not a recovery plan — a measured restore is.

LAB 7 — PRODUCTION READINESS

The course's final lab: treat training_store as a go-live candidate.

OFFICIAL LAB 7

60 MIN

labs/day-03/lab7/LAB-7-GUIDE.md

SCENARIO

Reload first if Day 2–3 writes remain. Never restore over training_store.

- 1 Stop the catalog collection scan
- 2 Monthly paid-order pipeline
- 3 Validator on training_store_ops
- 4 Restore to training_store_restore
- 5 Score go-live with evidence

YOUR TASKS

- 1 Filter, project, sort, limit, explain
- 2 Build the pipeline with \$match first
- 3 Prove one accepted and one rejected insert
- 4 Restore and compare product counts
- 5 Classify: ready, conditional or not ready

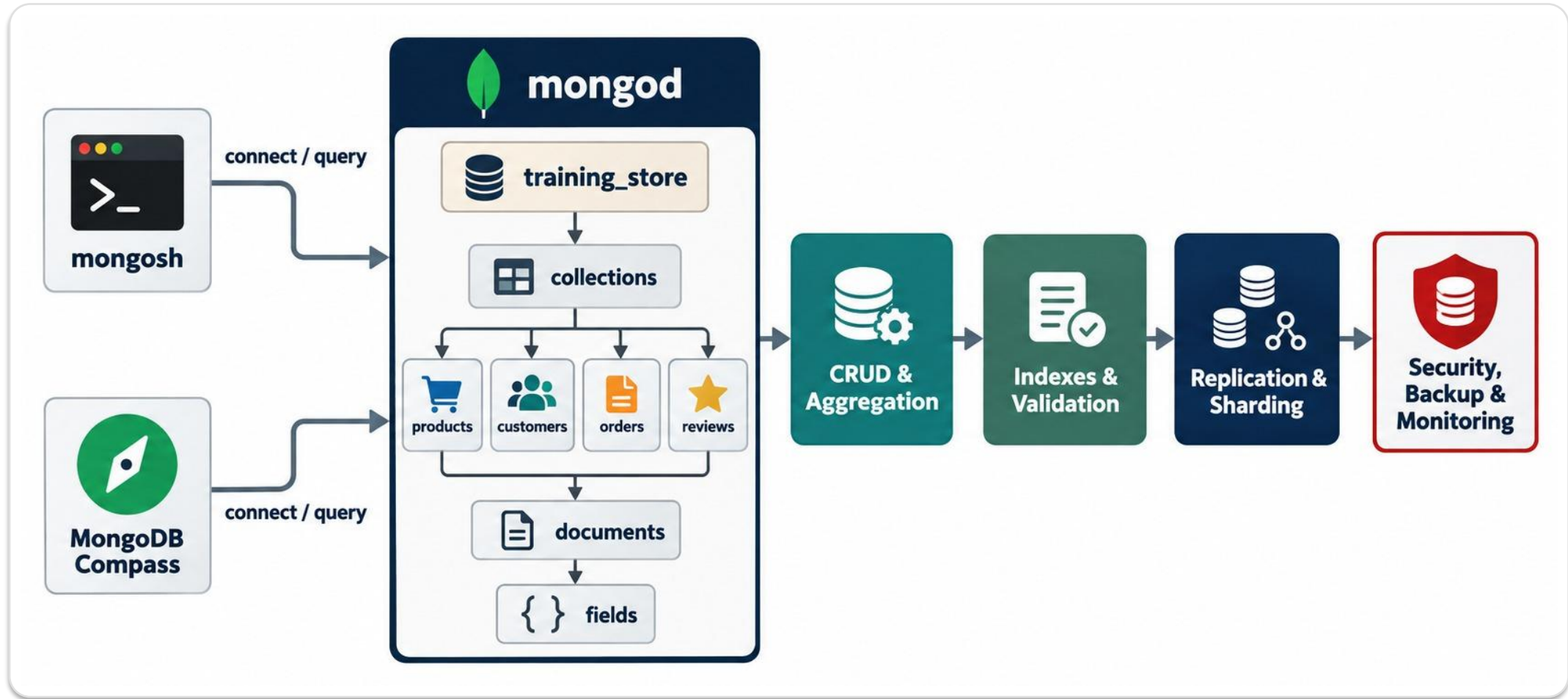
EXPECTED RESULTS

- ✓ Two active laptops returned
- ✓ Three months: July to September
- ✓ OPS-BAD rejected
- ✓ 13 products in both databases
- ✓ Not ready — with named evidence

KEY TAKEAWAY Lab 7 turns the whole course into one production-readiness review.

THE COURSE AT A GLANCE

One database, eight modules, one layer at a time.



KEY TAKEAWAY Every module added one layer to the same training_store database.

COURSE SUMMARY AND NEXT STEPS

From why NoSQL exists to running MongoDB securely and reliably.

YOU CAN NOW

- ✓ Choose a data model and design documents
- ✓ Install, connect and load MongoDB
- ✓ Query, update and aggregate documents
- ✓ Index from query shapes and explain
- ✓ Replicate for availability; shard for scale
- ✓ Secure, back up, monitor and troubleshoot

NEXT: APPLY IT TO YOUR OWN SYSTEMS

- 1 Which query will you explain first?
- 2 When will you test a restore?
- 3 Which alert needs a runbook?

TAKE WITH YOU The readiness checklist, your Lab 7 scorecard and one action for next week.

KEY TAKEAWAY Model around access patterns, measure before tuning, secure by default and prove every recovery.